Mémo carte Leaflet

Table des matières

1 vershtml 2
1.1 Arguments 2
1.2 Génération du source html 3
1.3 versgeojson 3
2 Création carte Leaflet 4
2.1 Colonnes spécifiques des vues 4Map 4
2.2 Afficher/masquer (griser) éléments de la légende à ouverture 4
2.3 Ordre des couches et libellés légende 5
2.4 Priorité d'affichage points, lignes, polygones 6
2.5 Ajustement taille bloc légende 6
2.6 Valeurs spécifiques traitées par le js 7
3 Debug 8
3.1 Origines des bugs 8
3.2 Types de bugs et action associée 8
3.3 Contrôles automatisés - script python 9
4 Exemples de génération 11
4.1 Lancement ps1 11
4.2 Lancement depuis pgadmin 11
4.3 Exemple d'une vue _4Map 11

1 vershtml

1.1 Arguments

Fonction postgres générant la carte Leaflet html
6 arguments : 1 obligatoire, 5 optionnels avec des valeurs par défaut
Renseigner obligatoirement '_nom_vue', format tableau de texte

Arguments

	Type de données	Mode	Nom de l'argument	Valeur par défaut
🗑	text[]	IN	_nom_vue	
🗑	text	IN	_caseacocher	'0'::text
🗑	text	IN	_titre	'Carte DGAML'::text
🗑	boolean	IN	_clic_zoom_actif	true
🗑	text	IN	_info_id	'vue_defaut'::text
🗑	text	IN	_rech_json	'false'::text

- _nom_vue : tableau de vues, syntaxe SQL : vershtml_n (array['vue1,'vue2', ...]
- Si _caseacocher n'est pas renseigné aucune couche n'est active à l'ouverture ('0' par défaut) : oeil barré et grisé dans sidebar. Si '1' => 1ere couche active, '1,1' : 2 1eres couches active, '1,0,1' : 1er et 3eme couche actives. Si on veut juste la 5eme, mettre : '0,0,0,0,1'.
  Concerne affichage des couches seulement pas les éléments légende **(*)**
- _info_id = valeur de la col 'id' de la table 'infos_carte' qui indique au code js de remplir le bloc Info par le contenu de la col. 'texte_info'. La f° ps1 'SI3P0-Ajouter-Information-Carte'remplit la table avec le texte info inclus dans le .ps1
- _rech_json est composé de 3 valeurs séparées par des « | »:
  1. n° couche à traiter = ordre de passage dans array['vue1, 'vue2', 'vue3', ...]
  2. filtre(s) de recherche = champ 'leafletsearch' ou nom de champ unique
  3. texte de l'invite de saisie, décrit l'objet recherché et les filtres

  On peut mettre plusieurs box de recherche en séparant par des « ; ». Ex.: -rechJson '1|leafletsearch|TMJA par RD, Année ; 4|RD |RD par n°'

  (*) selon la config et les manips il peut donc y avoir des éléments affichés en double sur la carte => **ne pas afficher des doublons dès l'ouverture** : choix couches actives vs. éléments de légende actifs

1.2 Génération du source html

Ligne de génération complète en fin de f° vershtml_n :
'return partieDébut || versgeojson_n (_nom_vue) || partieCodeJS'

- 1ere partie : appels aux api js, css
- 2eme partie : couches geojson générées par versgeojson_n() (**cf. 1.3)** et stockées dans un tableau js
- 3eme partie : cœur du code js traitant et affichant ces geojson

L'ensemble des fichiers js et css sont stockés en local. Cela évite les bugs suite à changements sur fichiers en ligne et permet les adaptations locales pour le CD30. En contrepartie, on ne bénéficie pas d'éventuelles améliorations => vérifier ponctuellement évolutions. Ex. au 06/26 v. 2.0 Leaflet beta : syntaxe js normalisée.

Il existe une f° vershtml_heatmap() calquée sur vershtml_n mais adaptée pour représenter des couches de chaleur (Ex : accidents RD Waze). Elle possède sa propre f° ps1 de lancement 'Parametrer-Job-SI3P0-Generer-Carte-Heatmap' avec les mêmes arguments que 'Parametrer-Job-SI3P0-Generer-Carte'.

1.3 versgeojson

La fonction vershtml_n() appelle une autre f° versgeojson_n() qui génère les couches geojson issues des vues passées en param dans '_nom_vue'.

- Conversion auto de tout format L93 en WGS84.
- Lignes et polygones transformés en ST_MultiLineString et ST_MultiPolygon
- Couches 2D/3D (3 coord x,y,z) passées en 2D via f° 'ST_Force2D'
- Lignes classées par ordre de grandeur décroissantes pour que les tronçons les plus petits soient en dernier dans le geojson donc dessinés en dernier, donc cliquables et surlignables (via switch) car situés par-dessus les plus grands.

2 Création carte Leaflet

Création de vues basées sur les tables et vues persistantes métiers issus en priorité du schéma « m »

2.1 Colonnes spécifiques des vues 4Map

- Chaque vue doit obligatoirement contenir une colonne '*geom'* contenant des coord. Les élts sans coord sont ignorés mais ceux renseignés s'affichent
- Le js détecte les vues vides (0 ligne ramenée) ou sans aucunes coord. et renomme/colore libellé en 'nom_vue vide'
- Le nom de la couche vient de la col. '*nomcouche'*. Si non renseigné, le js nomme en 'Couche 1', 'Couche 2' etc. Eviter d'utiliser des variables pr nom
- Pour les lignes, ajouter et remplir une colonne '*couleur'* dans la vue. Si colonne couleur absente, couleur par défaut = rouge
- Pour les points, ajouter et remplir une colonne '*icone'* dans la vue. Valeur : url vers l'icone. Si col. icone absente, icone par défaut = punaise bleue
- Pour ajouter une légende, ajouter et remplir une colonne '*legende'* dans la vue. Les valeurs constituent les libellés du bloc légende
- Ajouter col. bool 'dash' et mettre à true pour afficher lignes en pointillés. Ex : 'case when traitement = 'Ponctuel' then true else null end as dash'=> seules lignes à traitement 'ponctuel' mises en pointillés, les autres en traits pleins
- Les autres col. de la vue formeront le contenu des popups. Les champs sont ordonnés dans la fenêtre par longueur de nom de champ croissante => dans la vue, jouer sur les espaces au nommage pour les classer

Traitement js spécifique des champs et valeurs dans les popups :

- Champs « couleur | légende | icone | nomcouche | afficherlegouverture | leafletsearch | ordre_legende| dash » exclus des popups et tooltips
- Valeurs de champs vides de la vue mis à « Non défini »
- Champs commençant par « Lien » formatés en lien http actif et exclus des tooltips. Mettre url visibles ds tooltips en nommant autrement le champ.
- Popup spécifique pour tronçons superposés, 'Spiderfy' des points superposés.

2.2 Afficher/masquer (griser) éléments de la légende à ouverture

Par rapport aux couches de la sidebar (icone oeil), la légende distingue les éléments d'une même couche avec une vue adaptée (besoin métier legende à étudier avant) :

- créer col. ‘legende’ contenant les libellés des légendes. A chaque libellé légende distinct doit correspondre un icone ou couleur spécifique.
- par défaut, avec 1 col. ‘legende’ et 1 col. ‘icone’ ou ‘couleur’ renseignés, les éléments seront actifs dans la légende à ouverture et affichés sur la carte
- pour griser une couche contenant 1 col. legende, ajouter 1 col. ‘*AfficherLegOuverture’* la mettre à ‘false’ ou vide. Réactiver en mettant à ‘true’.
- pour griser une partie des éléments d’une couche, créer des filtres sql. Ex.
  - ‘’CodeVigilanceHydro > 1 as AfficherLegOuverture‘’ : type alerte >1 actives dans légende, celles = 1 grisées
  - ‘‘case when couleur = '#F29120' or couleur = '#449D67' then false else true end as AfficherLegOuverture‘’ : 2 éléments spécifiques de la couche grisés dans légende à ouverture, les autres activés (else true). On aurait pu référencer col.’ legende’ à la place de ‘couleur’ pour même résultat

Le nombre d’éléments par couche est indiqué entre parenthèses dans la sidebar. Si besoin, pour avoir le nombre d’éléments dans la légende, modifier la col. légende, ex. : <nom_champ> || ' (' || count(*) over (partition by <nom_champ>) || ')' as legende.

2.3 Ordre des couches et libellés légende

Ordre couches

Ordre de passage des vues/couches libre quel que soit leur type geom.
Les couches sidebar et bloc légende sont listées dans l’ordre de passage des vues.

Ordre libellés légende

- *Couche points* : les libellés légende suivent l’ordre défini dans la vue.
- *Couche lignes* : le js modifie l’ordre si col. num. ‘ordre_legende’ définie dans vue _4Map. Mettre valeurs voulues (1,2,3 …) puis faire un order by dessus.

  Ex. : ‘case CodeEtatAvancement3V when 1 then 'Tracé d''intention' when 2 then 'Etudes en cours' when 3 then 'Travaux en cours' when 4 then 'Ouvert' end as Legende, CodeEtatAvancement3V as ordre_legende , … order by ordre_legende ‘ :



  - si aucune col. ‘ordre_legende’ : le js classe par ordre alphabétique avec ordre intelligent des lignes (ex S1, S2, S10 au lieu de S1, S10, S2)

- éléments grisés placés par le js après les actifs par ordre alphabétique

NB : avec le tri alpha., utiliser quand ca fait sens (hiérarchie, classement), des astuces de nommage pour avoir l'ordre souhaité. Ex.: '1 – Niv. Structurant', '2 – Niv. de liaison', '3 – Niv. de proximité' sinon utiliser une col 'ordre_legende'.

Doublons de couleur/icone possibles dans une couche si libellés légende distincts.

2.4 Priorité d'affichage points, lignes, polygones

A ouverture, pour même type geom, une couche active dans le bloc légende s'affiche par-dessus une active dans la sidebar (icone œil vert).
A ouverture pour même type geom à l'intérieur de la sidebar ou de la légende, priorité d'affichage par ordre des vues (1ere prioritaire sur les autres, ordre descendant).

- Priorité entre types geom à l'ouverture et en cours de consultation:
  - le js priorise toujours l'affichage des points (icones) sur lignes/polygones
  - le js priorise toujours les lignes sur polygones. Ainsi lignes restent toujours cliquables sous un polygone même s'il vient d'être activé
- Priorité entre points :
  - à ouverture, le 1er élément légende s'affiche par-dessus les autres
  - en consultation, l'élément activé s'affiche par-dessus les autres
- Priorité entre lignes :
  - à ouverture, le dernier tronçon dessiné (du + grand au + petit cf. versgeojson) s'affiche par dessus les autres sur la carte (exception col. 'ordre_legende' cf. 2.3)
  - en consultation, l'élément activé s'affiche par dessus les autres

2.5 Ajustement taille bloc légende

Sur écran HD, pour éviter les débordements en hauteur, le bloc légende est :

- disposé sur 2 col. si nbr élts légende > 27
- réparti ensuite de façon équilibrée entre les 2 col.

Conséquence : éviter dès la conception un nbre total d'éléments légende > 55 (lisibilité ?). Regrouper les éléments sur 1 thème pour diminuer leur nombre ou activer la couche entière dans sidebar s'il est **inutile de distinguer ses éléments**

Remarques

- Le bloc légende est repliable en totalité ou par couches (chevrons)
- La taille de la fenêtre du navigateur joue aussi sur la bonne visibilité
- Le css revient à la ligne si libellé ou nom couche > 240px soit environ 50 caractères. Malgré cela, pour éviter de déborder en largeur et empiéter sur partie Est du Gard, **raccourcir** les libellés légende lors de la définition de la vue.
- Le plugin impression adapte carte et bloc selon hauteur légende (>400px): leger dézoom et décalage du Gard à l'ouest, diminution taille polices légende

2.6 Valeurs spécifiques traitées par le js

Selon la valeur de vue ou titre, le js va activer certaines parties de code :

- Si nom vue est dans ["d30 3vitinerairetype filtre","voies vertes","d30 3vitinerairetype","d30 troncon ref 3v sdm","d30 troncon ref 3v cd30", "reseauroutier","reseauroutier sanslegende","lineaire", ...] + champ 'Niveau' pour couleur RD : déclenchement affichage étiquettes dyn. Ajouter vues sans majuscules/accents dans var js « vuesEtiquetteRD » de vershtml
- Titre contient 'fauchage','curage','référentiel routier': surlignage à ouverture
- Titre 'Surveillance du r' : activation widget météo, refresh toutes les 10 min., luminosité fond N&B atténué, heure de maj et icone Waze dans titre
- Titre 'Surveillance du r' ou 'Limites de gestion PER' ou si sur 2 col pr 1 couche : pas de nom couche dans bloc légende pour aérer (codé dans leaflet.legend.js)
- Titre contenant 'UT de xxx' : centre et zoom sur l'UT concernée à ouverture
- Titre commençant par 'Graflex' : centre et zoom à ouverture, fond Ortho IGN
- Titre finissant par « 3D » : activation plugin visu bati 3D (pas utilisé au 09/26)

3 Debug

Tester modification du js d'abord en dur sur cartes tests du serveur Qualif.
Tester toute non régression : manips. utilisateurs, plugins, mix couches (pts,lignes).
Si OK reporter modifs dans f° vershtml_n.

3.1 Origines des bugs

Multiples mais le plus souvent :

- modif vershtml_n pas ou mal testé (effets de bord)
- mauvaise alim. de vue _4Map
- couche geojson incohérente **(*)**
- autres : syntaxe script ps1 KO, perturbations réseau CD30 ou SI tiers, source indispo ou déplacée, droit d'acces user svc_si3p0, etc.

(*) Origine du contenu 'anormal' technique ou fonctionnel d'1 couche geojson:

- **erreur de saisie manuelle à la source notamment sur les outils peu bornés style excel => éviter d'y recourir en interne ou en externe**
- évolution en amont (ex. chgt version) sur logiciels/APIs/chemins/formats pas annoncé par l'émetteur et qui désynchronise les scripts .ps1 d'import
- évolution du cadre légal impactant le référentiel et répercuté sur les données sources => anticiper les évolutions sur sites officiels, newsletters, forums, échanges inter-collectivités, gazette des communes…

NB : au 10/26, un message non bloquant s'affiche dans la console après sélection d'un fond maptiler : 'The cache API is only available in secure contexts' cela vient du site http://si3p0 non certifié (https). Si certificat interne dispo (certif *.gard.fr intranet), le lier au site renommé en si3p0.gard.fr. Points d'attention : inscrire le site au DNS local+ lien IIS, déploiement certif auto sur postes, vérif. expiration date.

3.2 Types de bugs et action associée

Carte ne se mettant plus à jour : origine possible (verif fichiers logs , console edge) :

- vue _4Map buggé/vide : corriger SQL et vérif sous pgadmin via un select
- script ps1 buggé : vérif. syntaxe, tabulations, retours lignes, mauvais C/C de lignes, ordre param, format param, chemins inexistants …
- script ps1 demandant une **interaction utilisateur** en cours d'exécution : question/validation sur un format, un chemin, une action, warning console

sous forme de message, etc. => tout le reste du script est bloqué lorsque lancé en tache programmée ce qui est le cas le plus courant dans si3p0

Carte plantée à ouverture ou après manipulation utilisateur :

- faire F12 dans Edge pour identifier bug et n° ligne, ajouter des console.log. Si cartes maj en même temps sont OK, cela ne vient pas du code js lui-même mais plus surement des vues/geojson sources
- analyser les couches geojson du source html, rechercher sur des mots clés : ''coordinates'', ''type'', « geometry », ''nomcouche'', ''legende'', etc.

Couche vide ou sans coord:

- sur carte, le js les signale en tant que couche vide : <nom vue>(0) en rouge
- identifier l'origine en remontant chaque étape. Ex.: vue vide => table 'm' vide ? => ps1 'moissonner' KO (logs) ? => accès aux sources KO ? => raison ?

Une carte plantée a « l'avantage » de faire apparaitre, après analyse, les sources à corriger.

Attention : un filtre SQL sur 1 vue/couche geojson peut débugger la carte mais aussi tronquer les infos visibles dessus => **corriger à la source en priorité**. Si impossible et qu'un filtre SQL ou js est appliqué, vérifier au cours du temps, la cohérence des infos.

3.3 Contrôles automatisés - script python

Système d'alertes automatisées des anomalies liées aux:

- fichiers Cartes .html
- fichiers Rapports .log
- tables taradata postgres
- *parsing des logs sur mots clés. Ex :'rollback', "error"=> risque de faux positifs*
- *parsing de logs par l'IA avec prompts? => pas de bot copilot dispo au 10/26*
- *Autre ?*

Un script python journalier tourne sur srw-si3p0 à 7h50 sauf les dimanches et vérifie :

1. tout fichier html des dossiers 'Cartes dynamiques' vide ou < 80 Ko
2. cartes journalières non mises à jour depuis **48 h.** Exclusion cartos d'autres fréquences (hebdo/mois/manuelle, exceptions) dans variable python
3. contrôle des geojson de toutes les cartes pour identifier celles contenant :
   - au moins une couche vide => "features": null ;
   - au moins un objet couche sans coord. => "geometry": null.
4. contrôles dans /SI3P0/Rapports des fichiers .logs :

- LàD_4h… du jour : absent ou < 1 Mo
- HorsBDDg_LàD_4h… du jour : absent ou < 100 Ko
- chaque lundi, D_4h… du dimanche (la veille donc) : absent ou < 200 Ko, seuil porté à 10 Mo le premier lundi du mois car taille moy. 35 Mo

5. Select sur vue f.taradata_log : alerte si date de mise a jour table <> date du jour. Csv de récap généré dans '\SI3P0-P0446\Rapports\csv ctrl fichiers\taradata_log_<date du jour>.csv'. Ctrl 'signalisation' inactif au 10/26.

Envoi d'une alerte mail auto si 1, 2, 4, 5 avec csv bilan en PJ.

Vérif manuelle des bilans d'alertes dans '\SI3P0-P0446\Rapports\csv ctrl fichiers\ rapport_cartes_dynamiques_<date du jour>.csv'.
Csv conservés sur 6 mois glissants.

4 Exemples de génération

4.1 Lancement ps1

Fonction '*Parametrer-Job-SI3P0-Generer-Carte' :* les noms des params ps1 diffèrent légèrement des noms des arguments sql mais sollicitent bien leur équivalent.
Paramètre '-carte' spécifique à la f° ps1 indiquant chemin/nom du fichier html

```
Parametrer-Job-SI3P0-Generer-Carte `
  -sources 'tmja_mensuel_global_4Map', 'ReseauRoutier_4Map', 'LimiteST_4Map'
  -couchesActives 'LimiteST_4Map' `
  -carte "$si3p0ThematiquesPortailWeb\Exploitation et trafic routier\Cartes
   dynamiques\Trafic moyen journalier annuel (TMJA).html" `
  -titre 'Trafic moyen journalier annuel (TMJA)' `
  -idInfo 'MJA5AnsComptageRoutier' `
  -rechJson '1|leafletsearch|TMJA par RD, Année, Commune, Type ; 2|RD   |RD' ` ))
```

Attention : si on ajoute 1 paramètre de lancement, matcher l'ordre des paramètres de 'Parametrer-Job-SI3P0-Generer-Carte' ⇔ 'vershtml_n()' (contrainte ps1)

Si ps1 abandonné, créer l'équivalent de la f° « SI3P0-Generer-Carte » dans le langage cible (python), recréer la connexion API à la base postgres et le mapping des params.

4.2 Lancement depuis pgadmin

Ordre de passage des arguments libres. Exemple :
'select vershtml_n (array['structuresrh', 'entretien_utal', 'limitegestion_ut'],
_caseacocher :='0,0,1', _rech_json:='1|Nom |structure RH par nom ;
2|leafletsearch|activité par RD, prad, date', _titre:='test – fauchage UT Ales' ;'

4.3 Exemple d'une vue _4Map

2 couches crées selon valeurs date récente/ancienne (cas particulier pour cette vue).
Alertes waze grisées à ouverture dans la légende sauf accident, véhicule, inondation.
Icones personnalisés attribués dynamiquement selon type d'alerte

```sql
create view AlerteWaze_4Map as
select
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alerte récente Waze' else 'Alerte ancienne Waze' end as "Nature ",
    to_char(a.DateCreation, 'DD/MM/YYYY HH24:mi')  as "Date de création ",
    p._NumeroRoute  as "RD               ",
    PRAEnTexte(p._PRA) as "PR+Abs             ",
    t.Description   as "Type               ",
    st.Description  as "Sous-type           ",
    a.Fiabilite  as "Fiabilité           ",
    'https://www.waze.com/live-map?lat=' || round(ST_Y(TransformerEnWGS84(a.Geom))::numeric, 6) || '&lon=' || round(ST_X(TransformerEnWGS84(a.Geom))::numeric, 6) as "Lien LiveMap                    ",
    case when (Age(Now(), DateCreation) < interval '6 hours') then 'Alertes récentes Waze' else 'Alertes anciennes Waze' end as NomCouche,
    CASE
        WHEN t.description LIKE '%Accident%'
            OR st.description LIKE '%Accident%'
            OR st.description IN ('Danger présence véhicule urgence', 'Inondations') THEN true
        ELSE false
    END AS AfficherLegOuverture,
    case
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when (Age(Now(), DateCreation) < interval '6 hours') and a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/couleur/' || a.IdTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze <> '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdSousTypeAlerteWaze || '.png'::varchar
        when a.IdSousTypeAlerteWaze = '' then '/Ressources/Images/Waze/70x70/gris/' || a.IdTypeAlerteWaze || '.png'::varchar
        else '/Ressources/Images/Waze/70x70/couleur/MISC.png'::varchar
    end as Icone,
    case
        when a.IdSousTypeAlerteWaze <> '' then st.Description
        else t.Description
    end as Legende,
    a.Geom
from AlerteWaze a
cross join PointVersPRA(Geom, 25) p
inner join TypeAlerteWaze t on a.IdTypeAlerteWaze = t.IdTypeAlerteWaze
inner join TypeAlerteWaze st on a.IdSousTypeAlerteWaze = st.IdTypeAlerteWaze
where a.IdTypeAlerteWaze not in ('JAM', 'ROAD_CLOSED') and a.IdSousTypeAlerteWaze <> 'HAZARD_WEATHER_FOG'
and a.Fiabilite > 6;
```